Plateforme intelligente de pilotage et d'analyse décisionnelle du CIAT

Tests, anomalies et corrections

ISEP Amadou Trawaré de Diamniadio · Centre d'Innovation, d'Application et de Transfert
Évaluation pratique en intelligence artificielle générative · Projet 3 · document généré le 03/11/2026

Phase 11 – Tests, anomalies et corrections

Objectif de la phase : vérifier chaque fonctionnalité exigée, consigner les anomalies et leurs corrections.

1. Synthèse

Type de test	Outil	Nombre	Résultat
Tests automatisés du backend (API + base PostgreSQL réelle)	pytest	80	✅ 80 réussis
Tests automatisés du frontend	Vitest + Testing Library	8	✅ 8 réussis
Contrôle de types du frontend	TypeScript (`tsc -b`)	—	✅ aucune erreur
Parcours de bout en bout dans un vrai navigateur (12 écrans × 2 tailles)	Playwright + Edge	24	✅ 0 erreur console, 0 défilement horizontal
Vérification visuelle des rapports PDF	rendu en image	2 rapports	✅ (2 corrections de mise en page, voir A3)

Commandes :

```
cd backend && python -m pytest          # 80 tests (base de test dédiée « ciat_vision_test »)
cd frontend && npm test                  # 8 tests
cd frontend && npm run lint              # contrôle de types
```

2. Couverture des exigences (prompt § 10)

Exigence	Tests (fichiers)
Connexion	`test_auth_permissions.py` : connexion, mauvais mot de passe, identifiant inconnu (même message), jeton invalide, blocage après 5 échecs, compte désactivé, renouvellement de session, changement de mot de passe ; `app.test.tsx` : formulaire, message d'erreur, enregistrement du jeton
Permissions	`test_auth_permissions.py` : l'Analyste ne peut ni créer, ni supprimer, ni importer ; seul l'administrateur administre ; coordonnées masquées pour l'Analyste ; l'administrateur ne peut pas se retirer ses droits
Création d'un projet	`test_projects.py` : création (porteur + candidature + projet), doublon refusé (insensible à la casse), validation des champs avec messages en français
Modification d'un projet	`test_projects.py` : modification ; cycle de vie complet avec transitions interdites, évaluation obligatoire, jalons automatiques, baisse de progression justifiée, « Terminé » = 100 %
Suppression d'un projet	`test_projects.py` : suppression logique, disparition des KPI, restauration par l'administrateur
Calcul des KPI	`test_kpi_charts.py` : **chaque indicateur est recalculé indépendamment en Python** et comparé au moteur SQL, sans filtre, par cohorte, par domaine, par statut, par période ; dénominateur nul → « non calculable » ; seuil de tolérance

Exigence	Tests (fichiers)
Filtres	`test_kpi_charts.py`, `test_projects.py` : filtres de l'API, recherche, pagination, statut inconnu refusé
Graphiques	`test_kpi_charts.py` : les séries des graphiques sont cohérentes avec les KPI (sommes identiques) ; capture de tous les graphiques sans erreur
Export PDF	`test_fundings_reports.py` : les 6 rapports en PDF valides
Export Excel	`test_fundings_reports.py` : les 6 rapports en Excel ; valeurs **identiques** au tableau de bord ; mention « données simulées »
Export CSV	`test_fundings_reports.py` : filtres appliqués, mention « données simulées »
Assistant IA	`test_assistant.py` : les 8 questions du cahier des charges en mode dégradé ; fonction non autorisée refusée ; aucune donnée personnelle dans les résultats ; résultats identiques au tableau de bord ; **boucle IA testée avec un client Claude simulé** (appel de fonction, transmission des résultats, erreur renvoyée au modèle pour correction)
Importation	`test_imports_admin.py` : colonne manquante, date invalide, statut non autorisé, domaine inconnu, doublon interne, doublon en base, refus sans confirmation, import partiel confirmé, Excel, format refusé
Financements	`test_fundings_reports.py` : reçu ≤ accordé ≤ demandé, statut cohérent avec les montants, dates ; financement obtenu calculé
Données simulées	`test_seed.py` : cohérence (évaluations avant décision, historique chronologique, règles métier) ; `test_zz_purge.py` : purge, disparition du bandeau, conservation des comptes

3. Performances mesurées

Mesures locales sur PostgreSQL 16, avec une base dédiée peuplée de **1 030 projets simulés**, soit la volumétrie de la cible du cahier des charges (§ 8.1). L'API a été appelée 20 fois par opération. Script reproductible : `backend/scripts/mesurer_performances.py`.

Opération	Médiane	95e centile	Cible	Résultat
Tableau de bord complet affiché dans le navigateur (Edge, 5 chargements)	1,18 s	1,80 s (max.)	< 2 s	✅
Page Analyses complète (12 graphiques) dans le navigateur	1,05 s	—	< 2 s	✅
Indicateurs du tableau de bord (API)	38 ms	44 ms	< 500 ms	✅
Indicateurs filtrés par cohorte	36 ms	42 ms	< 500 ms	✅
Séries de tous les graphiques d'analyse	126 ms	136 ms	< 500 ms	✅
Liste des projets (page de 20)	89 ms	107 ms	< 500 ms	✅
Recherche plein texte de projets	83 ms	89 ms	< 500 ms	✅
Projets en retard	154 ms	171 ms	< 500 ms	✅

Opération	Médiane	95e centile	Cible	Résultat
Liste et synthèse des financements	101 ms	111 ms	< 500 ms	✅
Assistant, projets à surveiller (mode dégradé)	82 ms	93 ms	< 30 s	✅
Rapport PDF global	0,45 s	—	< 10 s	✅
Rapport PDF des projets (1 030 lignes, le cas le plus lourd)	4,8 s	—	< 10 s	✅
Rapport Excel des projets	0,44 s	—	< 10 s	✅

Limites : mesures sur un poste local. Sur l'offre gratuite de Render, le serveur est moins puissant et le premier appel après une mise en veille est lent. Le temps de réponse de l'assistant en mode IA dépend du service Claude et n'a pas pu être mesuré sans clé d'API.

4. Journal des anomalies

N°	Anomalie constatée	Cause	Correction	Vérification
A1	Docker Desktop ne démarre pas sur le poste de développement	Moteur de virtualisation indisponible	PostgreSQL 16 portable (binaires officiels) lancé sur le port 5433 ; `docker-compose.yml` fourni pour les postes où Docker fonctionne	Migrations et 80 tests exécutés sur PostgreSQL 16.15
A2	Données simulées peu réalistes : 1 seul projet « en incubation », taux de sélection de 58 %	Tous les projets d'une cohorte démarraient le même jour ; notes trop élevées	Entrées en incubation échelonnées, durées variables, niveau des notes recalibré	44,5 % de sélection, tous les statuts représentés
A3	Rapport PDF : titre « Graphiques » isolé en bas de page ; étiquette longue tronquée	Titre non solidaire du premier graphique ; marge gauche insuffisante	Titre regroupé avec le premier graphique (`KeepTogether`) ; zone d'étiquettes élargie	Rendu PDF contrôlé visuellement
A4	Construction du frontend impossible : « Cannot apply unknown utility class btn »	Tailwind CSS 4 n'autorise pas `@apply` sur une classe de composant	Boutons déclarés avec `@utility`	`vite build` réussi
A5	**La page de l'assistant plantait après la première réponse** (« destroy_ is not a function »)	Un `useEffect` renvoyait le résultat de `scrollIntoView()`, qui est une promesse dans les navigateurs récents ; React l'a pris pour une fonction de nettoyage	Corps de l'effet entre accolades ; tous les effets du même type harmonisés	Parcours Playwright : 0 erreur ; question posée et réponse affichée
A6	Historique incohérent dans les données simulées : « Sélectionné » avant « En évaluation »	Date d'évaluation simulée parfois postérieure à la décision du comité	Évaluation toujours antérieure à la décision	Test automatique `test_seed.py` (0 incohérence)

N°	Anomalie constatée	Cause	Correction	Vérification
A7	Messages d'erreur du serveur en anglais (« There was an error parsing the body »)	Messages par défaut de Starlette	Gestionnaire d'erreurs HTTP traduisant les messages courants	Contrôle manuel
A8	Tableau de bord sur téléphone : valeurs des taux coupées sur deux lignes	Libellé et valeur côte à côte dans une carte étroite	Disposition verticale sur téléphone, valeur insécable ; taille des montants réduite	Nouvelle capture à 390 px
A9	Doublon de nom non détecté si les majuscules accentuées diffèrent (base locale en locale « C »)	`lower()` PostgreSQL et Python ne traitent pas les accents de la même façon	Comparaison faite des deux côtés par PostgreSQL (`lower()` / `lower()`)	Test de doublon insensible à la casse
A10	Premier chargement lourd (811 Ko de JavaScript)	Toutes les pages et Recharts dans un seul fichier	Chargement des pages à la demande (`React.lazy`)	Fichier initial : 296 Ko (95 Ko compressés)

5. Scénarios de validation manuelle (démonstration)

1. Se connecter en **analyste** : vérifier l'absence des boutons de modification et des menus Importation/Administration ; ouvrir une fiche projet : coordonnées masquées.
2. Se connecter en **coordonnateur** : créer un projet → « En attente » ; ajouter une évaluation → « En évaluation » ; le sélectionner ; le passer « En incubation » (5 jalons créés) ; marquer un jalon atteint ; ajouter un financement dont le montant reçu dépasse l'accordé → refus explicite.
3. Tableau de bord : filtrer sur une cohorte → les 8 indicateurs, les graphiques et le rapport PDF changent ensemble.
4. Assistant : poser les 8 questions du cahier des charges ; déplier « Données utilisées ».
5. Importation : déposer un fichier comportant une erreur et un doublon → prévisualisation, puis import des seules lignes valides après confirmation.
6. Se connecter en **administrateur** : consulter le journal (toutes les actions précédentes y figurent).